

FICHEIROS DE TEXTO E BINÁRIOS EXPORTAÇÃO DE DADOS E PERSISTÊNCIA

Ficheiros

- Os ficheiros permitem gravar dados no disco e reavê-los mais tarde, mesmo que o computador tenha sido desligado entre a operação de escrita e leitura.
- Podem gravar-se dados nos formatos:
 - Binário – filmes, músicas, executáveis, etc.
 - Texto – um arquivo de texto gravado com o notepad ou outro editor
- Além das classes que permitem gravar e ler ficheiros, temos classes auxiliares tais como:
 - DirectoryInfo
 - FileInfo
 - Path

DirectoryInfo

- Permite criar, mover e iterar sobre diretorias e subdiretorias
- No exemplo seguinte, usamos uma instância da classe para confirmar se a diretoria c:\temp existe, sendo esta criada caso não exista

```
var dirInfo = new DirectoryInfo(@"C:\temp");  
  
if (!dirInfo.Exists)  
    dirInfo.Create();
```

- E mostrar na consola, todas as suas subdiretorias

```
foreach (var dir in dirInfo.GetDirectories())  
    Console.WriteLine(dir);
```

DirectoryInfo

- Principais métodos

Métodos	Descrição
GetDirectories()	Permite obter uma coleção de subdiretórias
GetFiles()	Obtém uma coleção de todos os ficheiros
Delete()	Apaga a diretoria com a opção de apagar recursivamente todas as hierarquias de diretorias abaixo dessa.
Create()	Permite criar a diretoria especificada no momento da instanciação do objeto
CreateSubdirectory()	Permite criar uma subdiretoria dentro da diretoria
MoveTo()	Permite mover uma diretoria e o seu conteúdo para uma nova localização.
Exists()	Devolve true caso a diretoria exista

FileInfo

- Permite criar, copiar, mover e apagar ficheiros
- No exemplo seguinte, usamos uma instância da classe para confirmar se o ficheiro c:\temp\hello.txt existe, e se existir,

```
var fileInfo = new FileInfo(@"C:\temp\Hello.txt");  
  
if (fileInfo.Exists)  
    fileInfo.Delete();
```

- ... e se não existir, criamos um novo ficheiro no novo caminho como

```
if (fileInfo.Exists)  
    fileInfo.CopyTo(@"C:\temp\Ola.txt");
```

FileInfo

- Principais métodos e propriedades

Métodos	Descrição
Exists()	Permite verificar se o ficheiro existe
CopyTo()	Permite copiar o ficheiro para outro
Delete()	Apaga o ficheiro permanentemente

Propriedades	Descrição
FullName	Obtém o caminho completo para o ficheiro
Length	Obtém o tamanho do ficheiro, em bytes
Extension	Obtém a extensão do ficheiro
Name	Obtém o nome do ficheiro

Path

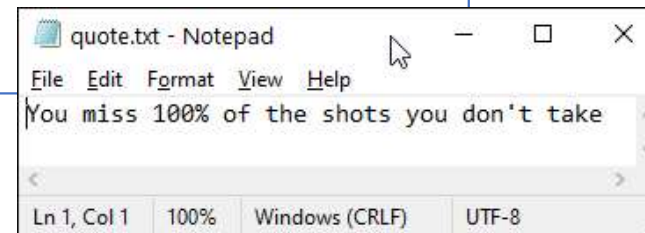
- A classe Path efetua operações com strings que representam informação sobre ficheiros e diretorias.
- Um dos métodos mais úteis é o Combine(), construindo o caminho completo para um ficheiro ou diretoria, com base em vários caminhos parciais e nomes de ficheiros ou diretorias.

```
var fileName = "hello.txt";  
var directory = @"C:\temp";  
  
string filePath = Path.Combine(directory, fileName);  
var fileInfo = new FileInfo(filePath);  
  
fileInfo.Create();
```

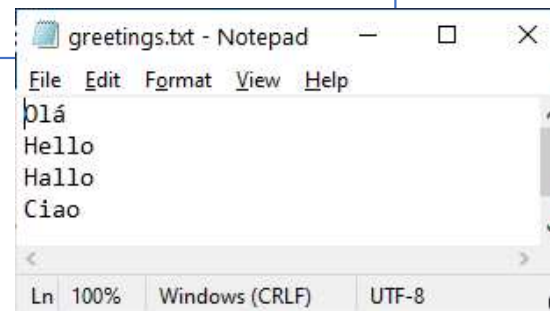
File

- Permite criar ou ler ficheiros de texto, entre outras operações

```
var path = @"C:\temp\quote.txt";  
var content = "You miss 100% of the shots you don't take";  
  
File.WriteAllText(path, content);
```



```
var path = @"C:\temp\greetings.txt";  
string[] greetings = { "Olá", "Hello", "Hallo", "Ciao" };  
File.WriteAllLines(path, greetings);
```



JSON (JavaScript Object Notation)

- É um [formato](#) compacto, de [padrão aberto](#) independente, de troca de dados simples e rápida (parsing) entre sistemas, especificado por [Douglas Crockford](#) em 2000, que utiliza texto [legível a humanos](#), no formato atributo-valor (natureza auto-descritiva)
- Isto é, um modelo de transmissão de informações no formato texto, muito usado em [web services](#) que usa transferência de estado representacional ([REST](#)) e aplicações [AJAX](#), substituindo o uso do [XML](#)
- Fonte: Wikipédia
<https://pt.wikipedia.org/wiki/JSON>

JSON (Exemplo)

```
public class Carro
{
    public string  Marca { get; set; }
    public string  Cor { get; set; }
}
```

```
var carro = new Carro() { Marca = "Ferrari", Cor = "Vermelho" };
```

- A representação do objeto carro em JSON seria:

```
{
  "Marca": "Ferrari",
  "Cor": "Vermelho"
}
```

Escrita de objetos em ficheiros usando JSON

1. Criação dos objetos
2. Serialização para o formato JSON (string)
3. Gravar a representação do objeto JSON em ficheiro



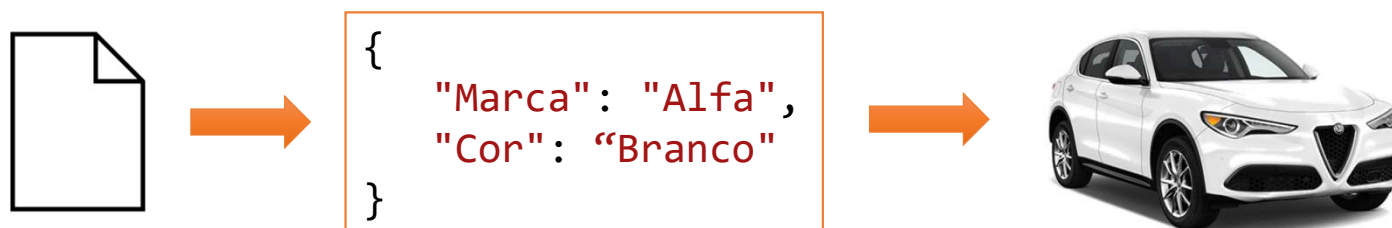
```
{  
  "Marca": "Alfa",  
  "Cor": "Branco"  
}
```



```
1) var carro = new Carro() { Marca = "Ferrari", Cor = "Vermelho" };  
2) string json = System.Text.Json.JsonSerializer.Serialize(carro);  
3) File.WriteAllText(@"C:\temp\cars.json", json);
```

Leitura de ficheiros em formato JSON

1. Leitura do ficheiro
2. Deserialização dos dados e criação automática dos objetos representados em JSON



```
var json = File.ReadAllText(@"C:\temp\cars.json");  
var carro = System.Text.Json.JsonSerializer.Deserialize<Carro>(json);
```

- **Nota:** É necessário que as classes tenham um construtor sem parâmetros, e especificar o tipo de objeto para o qual vamos fazer a deserialização

Conceitos Gerais

- O processo de manipulação de ficheiros efetua-se através de ***streams***
- ***Stream*** é um fluxo de informação que circula de um local para outro (p. ex., da RAM para dispositivos de armazenamento)
- A manipulação de ficheiros pode ser efetuado de duas formas: no formato de **texto** ou no formato **binário**
- Nos ficheiros de texto os dados são separados por delimitadores (ex: tabulação, | (pipe), ; (ponto e vírgula), virgula, etc.
- Nos ficheiros binários não são necessários delimitadores - o tipo de informação a registar ou a ler é determinado pela quantidade de bytes

Classes para manipulação de Streams

- **FileStream** – classe que define o modo de manuseamento de ficheiros (path, modo de abertura, modo de acesso)
- **BinaryReader** e **BinaryWriter** – classes para manipular ficheiros binários
- **StreamReader** e **StreamWriter** – classes para manipulação de ficheiros de texto
- Estas classes pertencem ao *namespace* System.IO

Modos de abertura e modos de acesso

- É realizado através de um objeto **FileStream**.
- O construtor impõe a seguinte assinatura :
FileStream(Path, FileMode, FileAccess)

FileMode	
Constante	Descrição
Append	Adiciona dados ao fim do ficheiro; cria o ficheiro se não existir; só para escrita
Create	Cria um novo ficheiro; se existir o ficheiro, este é substituído
CreateNew	Cria um novo ficheiro; se existir o ficheiro com o mesmo nome, é lançada um exceção
Open	Abre um ficheiro existente; se não existir é lançada uma exceção
OpenOrCreate	Abre um ficheiro existente; se não existir, cria um novo
Truncate	Abre um ficheiro existente; Elimina todo o conteúdo do ficheiro

FileAccess	
Constante	Descrição
Read	Acesso apenas para leitura
ReadWrite	Acesso para leitura e escrita
Write	Acesso apenas para escrita

Gravar um ficheiro Binário

- `// recolher informação`
- `string Nome = tbNome.Text;`
- `int Idade = int.Parse(nmIdade.Value.ToString());`
- `string Profissao = tbProfissao.Text;`
- `// Definir o stream de ligação ao ficheiro; neste caso em modo append e acesso de escrita`
- `FileStream fs = new FileStream(caminho, FileMode.Append, FileAccess.Write);`
- `// criar o buffer binário de escrita`
- `BinaryWriter bw = new BinaryWriter(fs);`
- `// Guardar os dados no ficheiro`
- `bw.Write(Nome);`
- `bw.Write(Idade);`
- `bw.Write(Profissao);`
- `// Fechar o ficheiro e o stream`
- `bw.Close();`
- `fs.Close();`

Ler um ficheiro Binário

```
• // verificar se o ficheiro existe
if (File.Exists(caminho))
{
    // Criar um stream de ligação ao ficheiro em modo leitura
    •FileStream fs = new FileStream(caminho, FileMode.Open, FileAccess.Read);
    // Definir o buffer binário de leitura
    •BinaryReader br = new BinaryReader(fs);
    // Verificar se ainda existe byte para ler
    •while((br.PeekChar() != -1))
    •{
        // ler informação do ficheiro
        •string Nome = br.ReadString();
        •int Idade = br.ReadInt32();
        •string Profissao = br.ReadString();

        // utilizar os dados ...
    •}
    // Fechar o buffer e o stream
    •br.Close();
    •fs.Close();
•}
```

Gravar um ficheiro de texto

- `// recolher informação`
- `string Nome = tbNome.Text;`
- `int Idade = int.Parse(nmIdade.Value.ToString());`
- `string Profissao = tbProfissao.Text;`
- `// Definir o stream de ligação ao ficheiro em modo append e para escrit`
- `FileStream fs = new FileStream(caminho, FileMode.Append, FileAccess.Write);`
- `// Criar o buffer de texto de escrita`
- `StreamWriter sw = new StreamWriter(fs);`
- `// Guardar os dados no ficheiro com string delimitados por ;`
- `sw.WriteLine(Nome + ";" + Idade + ";" + Profissao);`
- `// Fechar o ficheiro e o stream`
- `sw.Close();`
- `fs.Close();`

Ler um ficheiro de texto

```
• // verificar se o ficheiro existe
• if (File.Exists(caminho))
{
    •// Criar um stream de ligação ao ficheiro em modo leitura
    •FileStream fs = new FileStream(caminho, FileMode.Open, FileAccess.Read);
    •// Definir o buffer de leitura do ficheiro de texto
    •StreamReader sr = new StreamReader(fs);
    •while((sr.Peek() != -1)) // Verificar se ainda existe carater para ler
    •{
        •string linha = sr.ReadLine(); // lê uma linha
        •int pos1 = linha.IndexOf(";") + 1; // Apanha a posição dos delimitadores
        •int pos2 = linha.IndexOf(";", pos1) + 1;
        •string Nome = linha.Substring(0, pos1 - 1); // ler informação do ficheiro
        •int Idade = Convert.ToInt32(linha.Substring(pos1, pos2-pos1-1));
        •string Profissao = linha.Substring(pos2);

        •// utilizar os dados ...
    •}
    •sr.Close(); // Fechar o buffer e o stream
    •fs.Close();
•}
```

Serialização (Serializale)

- A serialização é o processo de converter um objeto num fluxo de bytes;
- O objetivo é armazená-lo para ser enviado para a memória, uma base de dados ou um ficheiro.
- A sua finalidade principal é salvar o estado de um objeto para mais tarde o poder recriar.
- O processo inverso é chamado desserialização (Deserializale).

Serialização (Serialize)

- O processo de serializar ou desserializar objetos, para serem guardados num ficheiro, é realizado através da classe **BinaryFormatter** e dos respetivos métodos **Serialize()** e **Deserialize()**, em conjunto com um **Stream**;
- **Atenção:** para que os objetos possam ser “serializados” é necessário, definir na classe esse atributo:
 - **[Serializable]**
 - ```
class Cliente {
```
  - ```
    //Atributos
```
 - ```
 public String _Nome;
```
  - ```
    public String _NIF;
```
 - ```
 private String _Observacao;
```
  - ```
}
```

Serialização (Serializale)

```
//Criar uma novo cliente
novoCliente = new Cliente(tbNome.Text, tbNIF.Text, tbObservacao.Text);
FileStream stream = new FileStream(ficheiro, FileMode.Create, FileAccess.Write);
//Permite serializar/deserializar objectos para/de formato binário
BinaryFormatter bin = new BinaryFormatter();
//Serialize do Objecto para a Stream no formato binário
bin.Serialize(stream, novoCliente);
//fecha a stream
stream.Close();
```

```
[Serializable]
class Cliente {
    //Atributos
    public String _Nome;
    public String _NIF;
    private String _Observacao;
}
```

DESSerialização (DEserializale)

```
//Obter o ficheiro e abrir para uma Stream para ser interpretado
FileStream stream = new FileStream(ficheiro, FileMode.Open, FileAccess.Read);

//Criar o objecto para interpretar dados binários
BinaryFormatter bin = new BinaryFormatter();

//Deserializa o conteúdo para o objecto do tipo Cliente
novoCliente = bin.Deserialize(stream) as Cliente;

//fecha a Stream
stream.Close();

//Preenche os campos do formulário
tbNome.Text = novoCliente.Nome;
tbNIF.Text = novoCliente.NIF;
tbObservacao.Text = novoCliente.Observacao;
```

```
[Serializable]
class Cliente {
    //Atributos
    public String _Nome;
    public String _NIF;
    private String _Observacao;
}
```

Nota: BinaryFormatter foi descontinuado a partir do .NET9 por questões de segurança

Caixas de Diálogo – OpenFileDialog

```
// Configure open file dialog box
OpenFileDialog dlg = new OpenFileDialog();
dlg.FileName = "Document"; // Default file name
dlg.Filter = "txt files (*.txt)|*.txt|All files (*.*)|*.*"; // Default file
extensions
dlg.FilterIndex = 1; // Default filter index

// Show save file dialog box e process save file dialog box results
if (dlg.ShowDialog() == DialogResult.OK)
{
    {
        // Open document
        string filename = dlg.FileName;
        // ...
    }
}
```

- Só permite escolher ficheiros existentes
- Utilizado para ler um ficheiro, validando a existência do mesmo

Caixas de Diálogo – SaveFileDialog

```
// Configure save file dialog box
SaveFileDialog dlg = new SaveFileDialog();
dlg.FileName = "Document"; // Default file name
dlg.Filter = "txt files (*.txt)|*.txt|All files (*.*)|*.*"; // Default file
extensions
dlg.FilterIndex = 1; // Default filter index

// Show save file dialog box e process save file dialog box results
if (dlg.ShowDialog() == DialogResult.OK)
{
    // Save document
    string filename = dlg.FileName;
    // ...
}
```

- Ficheiro não tem de existir, permitindo a escrita de um novo nome
- Usado para escrita e pode alterar ficheiros existentes

MenuS em aplicação Windows forms

- Para adicionar um menu utilizamos o **MenuStrip**
 - Arrastar para o Design
 - Adicionar o menu, clicar e editar cada posição do menu
- Os eventos em cada item do menu funcionam da mesma forma que com outros componentes:
 - 2 cliques ou aceder a Events
 - Também se pode escolher eventos de outros componentes

